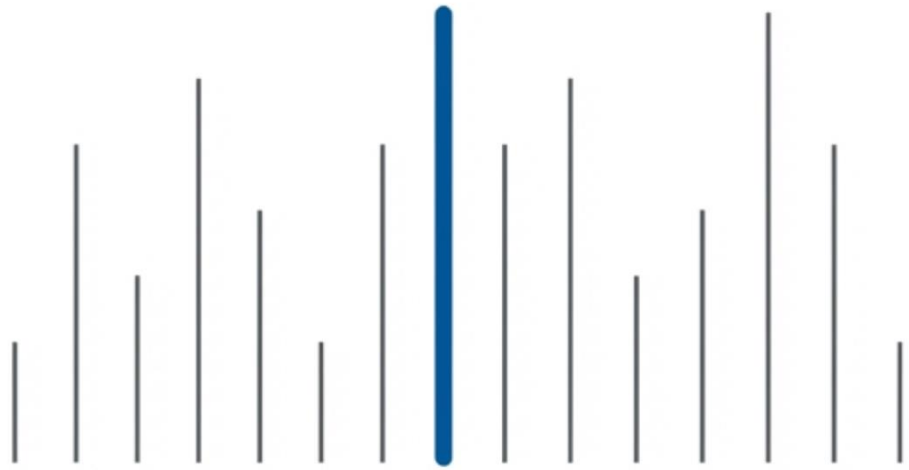
Universidad del Rosario | Escuela de Ciencias e Ingeniería | MACC

Algoritmos de ordenamiento:

Bubble sort, insertion sort y selection sort



Objetivos

- Entender las operaciones fundamentales (comparación e intercambio) que definen a los algoritmos *Bubble Sort*, *Insertion Sort* y *Selection Sort*.

- Aplicar el concepto de **invariante de bucle** (inicialización, mantenimiento y terminación) para probar matemáticamente que los algoritmos producen una salida ordenada.

- Diseñar e implementar variaciones de estos algoritmos para estructuras de datos con acceso iterativo restrictivo. Específicamente, entender cómo la ausencia de iteradores de acceso aleatorio (*Random Access Iterators*) en listas doblemente enlazadas exige la manipulación directa de punteros o iteradores bidireccionales (*Bidirectional Iterators*), a diferencia de la indexación directa en arreglos dinámicos.

- Identificar cuáles de estos algoritmos preservan el orden relativo de elementos con claves idénticas y por qué esta propiedad es crítica en sistemas de bases de datos y procesamiento de registros múltiples.

Motivación

El factor constante y el umbral de N pequeño: Asintóticamente, $n \log n$ domina a n^2. Pero en la práctica, para conjuntos de datos pequeños (típicamente $n < 16$ o $n < 32$), el bajo *overhead* (sobrecarga) de las operaciones de un *Insertion Sort* supera la recursión y asignación de memoria de algoritmos más complejos. Por esta razón, implementaciones industriales como std::sort en C++ (usualmente *Introsort*) cambian automáticamente a *Insertion Sort* cuando las particiones se vuelven pequeñas.

Localidad de Referencia y Jerarquía de Caché: La notación Big-O asume el modelo RAM (Random-Access Machine), donde cada acceso a memoria cuesta $O(1)$ constante. En el hardware moderno, un acceso a la memoria principal (RAM) puede costar cientos de ciclos de reloj, mientras que acceder a la caché L1 cuesta apenas un par de ciclos.

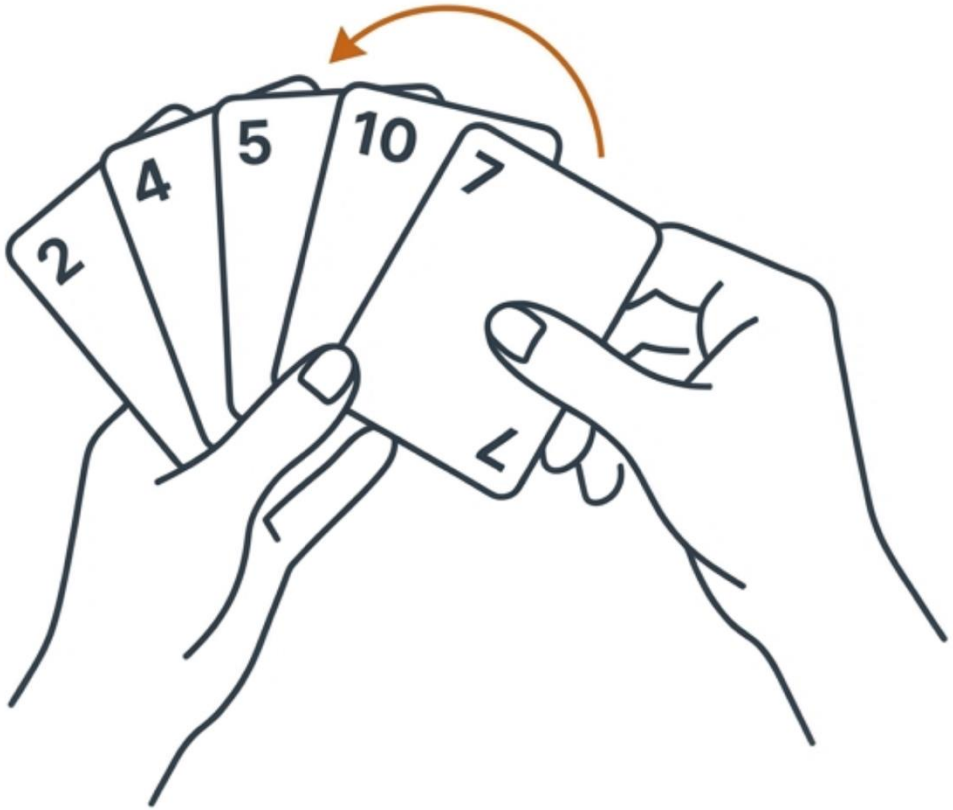
std::sort

Análisis de Topología de Memoria (C++20)

```
#include <iostream>
#include <vector>
#include <list>
#include <iomanip>

int main() {
    std::vector<int> v = {10, 20, 30};
    std::list<int> l = {10, 20, 30};

    std::cout << "--- Disposición en Memoria: std::vector ---\n";
    for (size_t i = 0; i < v.size(); ++i) {
                std::cout << "v[" << i << "] en dir: " << &v[i] << "\n";
    }

    std::cout << "\n--- Disposición en Memoria: std::list ---\n";
    int index = 0;
    for (auto it = l.begin(); it != l.end(); ++it, ++index) {
                std::cout << "l[" << index << "] dato en dir: " << &(*it) << "\n";
    }

    return 0;
}
```

Insertion-sort

- El método incremental: Ordenamos como si estuviéramos organizando una mano de cartas.
- Comenzamos con la mano izquierda vacía y las cartas boca abajo en la mesa.
- Tomamos una carta a la vez de la mesa y la insertamos en su posición correcta en la mano izquierda, comparándola de derecha a izquierda con las cartas ya ordenadas.

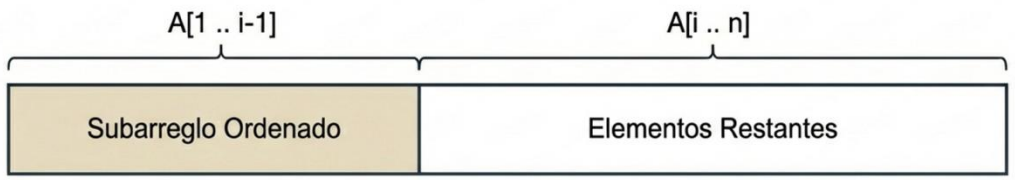


Insertion sort: Pseudo-código

```
INSERTION-SORT(A, n)
for i = 2 to n
    key = A[i]
    // Insertar A[i] en el subarreglo ordenado A[1 : i-1].
    j = i - 1
    while j > 0 and A[j] > key
        A[j+1] = A[j]
        j = j - 1
    A[j+1] = key
```

Invariante de Bucle y exactitud

```
INSERTION-SORT(A, n)
1 for i = 2 to n
2     key = A[i]
3     // Insertar A[i] en el subarreglo ordenado A[1 : i-1].
4     j = i - 1
5     while j > 0 and A[j] > key
6         A[j+1] = A[j]
7         j = j - 1
8     A[j+1] = key
```



Definición del Invariante para Insertion Sort:

Al inicio de cada iteración del bucle for (línea 1), el subarreglo A[1 : i-1] consiste en los elementos que originalmente estaban en A[1 : i-1], pero ahora en orden ordenado.

Invariante de Bucle y exactitud

```
INSERTION-SORT(A, n)
1 for i = 2 to n
2     key = A[i]
3     // Insertar A[i] en el subarreglo ordenado A[1 : i-1].
4     j = i - 1
5     while j > 0 and A[j] > key
6         A[j+1] = A[j]
7         j = j - 1
8     A[j+1] = key
```

Inicialización (Caso Base): Justo antes de la primera iteración, `i = 2`. El subarreglo evaluado es `A[1 : 1]`, que contiene un único elemento (el `A[1]` original). Un conjunto de un solo elemento está, por definición matemática, trivialmente ordenado. El invariante se cumple.

Mantenimiento (Paso Inductivo): Durante una iteración `i`, el bucle `while` desplaza los elementos `A[i-1]`, `A[i-2]`, etc., una posición a la derecha hasta encontrar la posición correcta para `A[i]` (la `key`). Al insertar la `key` en la línea 8, el subarreglo `A[1 : i]` contiene los mismos elementos originales, pero ahora en orden. Al incrementar `i` para la siguiente iteración, el invariante se preserva.

Terminación: El bucle `for` finaliza cuando `i` excede a `n` (es decir, `i = n + 1`). Sustituyendo `i` por `n + 1` en el enunciado del invariante, concluimos que el subarreglo `A[1 : n]` contiene los elementos originales en orden ordenado. Como `A[1 : n]` es la totalidad del arreglo, el algoritmo es **correcto**.

Selection Sort

```
SELECTION-SORT(A, n)
for i = 1 to n - 1
    min_idx = i
    for j = i + 1 to n
        if A[j] < A[min_idx]
            min_idx = j
    swap A[i] with A[min_idx]
```

El diseño algorítmico, formulado para un arreglo `A` de `n` elementos (indexado en 1), se estructura de la siguiente manera. Nótese que el bucle principal solo necesita iterar hasta `n - 1`

Invariante de Bucle y exactitud

```
SELECTION-SORT(A, n)
1 for i = 1 to n - 1
2     min_idx = i
3     for j = i + 1 to n
4         if A[j] < A[min_idx]
5             min_idx = j
6     swap A[i] with A[min_idx]
```

$A[1..i-1]$	$A[i..n]$
Ordenado y contiene los mínimos absolutos	Elementos restantes no procesados

Definición del Invariante:
Al inicio de cada iteración del bucle `for` externo (línea 1), el subarreglo `A[1 : i-1]` contiene los `i - 1` elementos más pequeños del arreglo original `A`, y están en orden ordenado.

Invariante de Bucle y exactitud

```
SELECTION-SORT(A, n)
1 for i = 1 to n - 1
2     min_idx = i
3     for j = i + 1 to n
4         if A[j] < A[min_idx]
5             min_idx = j
6     swap A[i] with A[min_idx]
```

Inicialización: Antes de la primera iteración, `i = 1`. El subarreglo `A[1 : 0]` está vacío. Un conjunto vacío contiene trivialmente los 0 elementos más pequeños de `A` y está ordenado. El invariante se cumple.

Mantenimiento: Durante la iteración `i`, el algoritmo examina el subarreglo `A[i : n]` para encontrar el elemento mínimo absoluto restante (líneas 2-5). Al intercambiar este mínimo con `A[i]` (línea 6), el subarreglo `A[1 : i]` ahora contiene los `i` elementos más pequeños de `A` en orden ordenado. Al incrementar `i` para la siguiente iteración, el invariante se preserva.

Terminación: El bucle finaliza cuando `i = n`. Sustituyendo en el invariante, el subarreglo `A[1 : n-1]` contiene los `n - 1` elementos más pequeños de `A` en orden ordenado. Por axioma matemático, el único elemento restante en `A[n]` debe ser obligatoriamente el elemento máximo (o mayor o igual a los anteriores). En consecuencia, todo el arreglo `A[1 : n]` está ordenado. La correctitud queda demostrada.

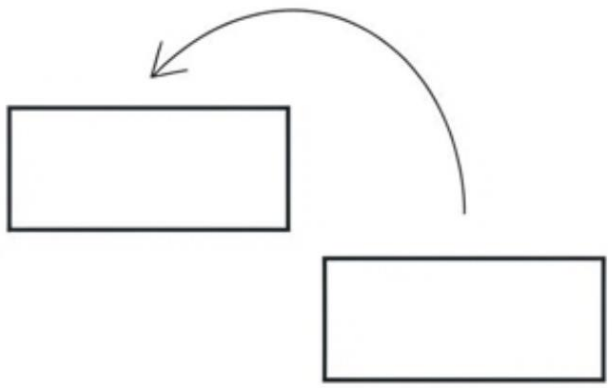
Bubble sort

```
BUBBLE-SORT-OPTIMIZED(A, n)
swapped = true
i = 1
while swapped
    swapped = false
    for j = 1 to n - i
        if A[j] > A[j + 1]
            swap A[j] with A[j + 1]
            swapped = true
    i = i + 1
```

A diferencia de la versión ingenua puramente cuadrática, presentaremos la versión optimizada que detecta si el arreglo ya está ordenado, permitiendo un mejor caso de $O(n)$.

Invariante de Bucle y exactitud

```
BUBBLE-SORT-OPTIMIZED(A, n)
1 swapped = true
2 i = 1
3 while swapped
4     swapped = false
5     for j = 1 to n - i
6         if A[j] > A[j + 1]
7             swap A[j] with A[j + 1]
8             swapped = true
9     i = i + 1
```



Definición del Invariante (Bucle Externo):

Al inicio de la iteración `i` *del bucle* `while` *(línea 3), el subarreglo* `A[n - i + 2 : n]` *contiene los* `i - 1` *elementos más grandes del arreglo original, y están en su posición final ordenada.*

Invariante de Bucle y exactitud

```
BUBBLE-SORT-OPTIMIZED(A, n)
1 swapped = true
2 i = 1
3 while swapped
4     swapped = false
5     for j = 1 to n - i
6         if A[j] > A[j + 1]
7             swap A[j] with A[j + 1]
8             swapped = true
9     i = i + 1
```

Inicialización: Antes de la primera iteración (i = 1), el subarreglo A[n + 1 : n] está vacío. Triviamente, contiene los 0 elementos más grandes en su posición final. El invariante se cumple.

Mantenimiento: Durante la iteración i, el bucle for interno (líneas 5-8) compara elementos adyacentes desde j = 1 hasta n - i. Si un elemento es mayor que su sucesor, se intercambian. Este proceso empuja inexorablemente el elemento máximo del subarreglo A[1 : n - i + 1] hacia el índice n - i + 1. Al finalizar la pasada e incrementar i (línea 9), el subarreglo ordenado al final del arreglo crece en un elemento, preservando el invariante.

Terminación: El bucle finaliza por dos razones. Si i = n, el subarreglo A[2 : n] contiene los n - 1 elementos mayores en su posición final, lo que implica que A[1] es el mínimo absoluto y el arreglo está ordenado. Alternativamente, si swapped == false al final de una iteración, significa que el bucle interno no realizó ningún intercambio; es decir, $A[j] \leq A[j+1]$ para todo j en el rango evaluado. Esto es la definición matemática de un arreglo ordenado, probando la correctitud de la salida anticipada.

Resumen

Algoritmo	Mejor Caso (Comparaciones)	Peor Caso (Comparaciones)	Mutaciones Máximas (Escrituras)	Estabilidad	Óptimo para...
Insertion Sort	O(n)	O(n2)	O(n2)	Estable	Arreglos pequeños o casi ordenados (alta localidad).
Selection Sort	O(n2)	O(n2)	O(n)	Inestable*	Memorias de estado sólido o sistemas donde la escritura es costosa.
Bubble Sort	O(n)	O(n2)	O(n2)	Estable	Fines puramente pedagógicos.